



ĐẠI HỌC BÁCH KHOA – ĐHQG TP.HCM

Khoa Khoa học và Kỹ thuật Máy tính

NGÂN HÀNG CÂU HỎI PHẢN BIỆN OS SIMULATOR

Kịch bản trả lời, bằng chứng code và giới hạn cần thừa nhận

Mục tiêu: không học thuộc output. Mỗi câu trả lời phải đi theo chuỗi
định nghĩa → cơ chế trong code → input/output chứng minh → giới hạn.

Môn học: Hệ điều hành – CO2018
Lớp: L08
Nhóm: K24 Warriors
GVHD: Thầy Hoàng Lê Hải Thanh

TP. Hồ Chí Minh, tháng 06/2026

Contents

1	Cách sử dụng tài liệu khi chuẩn bị demo	2
1.1	Khung trả lời bốn bước	2
1.2	Ba nguyên tắc sống còn	2
2	Kiến trúc tổng thể và vòng đời process	2
3	Scheduler, MLQ và thay đổi số CPU	3
4	Timer thread, concurrency và race condition	4
5	Logic READ, WRITE và MMU	5
6	ALLOC, FREE, KMALLOC và kernel cache	6
7	Syscall, wrapping và user/kernel protection	7
8	Paging 64-bit, canonical address và swap	7
9	Input, output, assertions và thay đổi trực tiếp	8
9.1	Các tình huống thầy có thể yêu cầu sửa tại chỗ	9
10	Câu hỏi về quá trình làm và sử dụng AI	10
11	Các câu hỏi khó và cách thừa nhận giới hạn	11
12	Bảng mở source trong 10 giây	11
13	Bộ câu hỏi luyện nhanh cho từng thành viên	12
13.1	Bảo Anh – kiến trúc và kiểm chứng	12
13.2	Quốc Trung – scheduler và synchronization	12
13.3	Quốc Bảo – memory và copy	13
13.4	Quốc An – syscall và protection	13
13.5	Trường Giang – MM64, paging và swap	13
14	Checklist trước khi bước vào phòng demo	13
15	Cách xử lý khi bị hỏi ngoài kịch bản	14

1. Cách sử dụng tài liệu khi chuẩn bị demo

Các thông tin từ nhóm đã báo cáo trước cho thấy thầy có thể yêu cầu chạy code, đổi input ngay tại chỗ, giải thích output, chỉ trực tiếp logic hàm, hỏi phần user/kernel, MMU, paging, scheduler, swap và cách nhóm kiểm chứng phần có hỗ trợ từ AI. Vì vậy, mục tiêu của tài liệu này không phải tạo “đáp án thuộc lòng”, mà là giúp nhóm trả lời có cấu trúc và trung thực.

1.1. Khung trả lời bốn bước

1. **Định nghĩa:** nói khái niệm bằng một hoặc hai câu.
2. **Cơ chế trong code:** nêu đúng file và hàm giữ property.
3. **Bảng chứng:** giải thích input cô lập điều gì và output nào cần quan sát.
4. **Giới hạn/trade-off:** nói rõ điều test chưa chứng minh hoặc thiết kế còn đơn giản.

Ví dụ ngắn:

“Weighted slot là số lần một priority queue được chọn trong một chu kỳ. Trong `get_mlq_proc()`, mỗi lần dequeue sẽ giảm `slot[prio]`. Test `sched_out_of_slot` cho thấy queue 137 được chọn ba lần trước khi chuyển queue. Tuy nhiên test này không chứng minh scheduler không starvation trong mọi workload vô hạn.”

1.2. Ba nguyên tắc sống còn

- Không khẳng định “test pass nghĩa là không còn bug”. Chỉ nói test chứng minh các property đã assert.
- Không nói swap hoàn chỉnh end-to-end nếu chưa tạo được demo rõ ràng. Nói code path nào có và điều gì chưa chứng minh.
- Khi không chắc một chi tiết, mở code rồi giải thích. Không đoán.

2. Kiến trúc tổng thể và vòng đời process

Câu hỏi: Simulator này có phải là một hệ điều hành thật không?

Trả lời đề xuất. Không. Đây là một chương trình C chạy trong user space. Nó dùng các `pthread` thật để mô phỏng loader, timer và các CPU; còn process mô phỏng chỉ là các cấu trúc `pcb_t`, không phải process thật của Linux/macOS.

Mở code/bảng chứng. `src/os.c` tạo loader/CPU thread; `include/common.h` định nghĩa `pcb_t`.

Giới hạn cần nói thẳng. Không có privilege level, interrupt phần cứng hoặc context switch thật. Các khái niệm đó được mô hình hóa bằng hàm và cấu trúc dữ liệu.

Câu hỏi: Giải thích luồng từ input đến khi process hoàn thành.

Trả lời đề xuất. Configuration quy định quantum, số CPU, số process, thời điểm load và priority. Loader đọc process file, tạo PCB rồi gọi `add_proc`. CPU gọi `get_proc`, thực thi từng instruction bằng `run`. Hết quantum thì `put_proc`; khi `pc == code->size` thì process được giải phóng.

```
config -> loader/load() -> PCB -> add_proc() -> get_proc() -> run() ->
put_proc()/finish
```

Mở code/bảng chứng. `src/os.c: read_config(), ld_routine(), cpu_routine(); src/loader.c: load(); src/cpu.c: run().`

Câu hỏi: PCB lưu gì và vì sao cần PCB?

Trả lời đề xuất. PCB lưu PID, priority, đường dẫn, code segment, program counter, registers, con trỏ kernel và cấu trúc memory. Nó cho phép CPU tạm dừng process khi hết quantum rồi tiếp tục đứng trạng thái ở lần dispatch sau.

Mở code/bảng chứng. `struct pcb_t` trong `include/common.h`.

Câu hỏi: Program counter thay đổi ở đâu?

Trả lời đề xuất. Trong `run()`, CPU lấy instruction tại `code->text[pc]` rồi tăng `pc`. CPU routine kiểm tra `pc == code->size` để xác định process đã hoàn thành.

Mở code/bảng chứng. `src/cpu.c: run()` và `src/os.c: cpu_routine()`.

3. Scheduler, MLQ và thay đổi số CPU

Câu hỏi: MLQ của bài hoạt động như thế nào?

Trả lời đề xuất. Có 140 ready queue, index từ 0 đến 139. Scheduler quét từ index nhỏ đến lớn, nên số priority nhỏ được xét trước. Trong mỗi queue, process được dequeue theo FIFO. Mỗi queue còn có quota `slot[prio] = MAX_PRIO - prio`.

Mở code/bảng chứng. `src/sched.c: init_scheduler(), get_mlq_proc()`.

Câu hỏi: Phân biệt priority, weighted slot, quantum và FIFO.

Trả lời đề xuất. **Priority** chọn queue nào được xét trước. **Weighted slot** là số lần queue được chọn trong một chu kỳ. **Quantum** là số instruction tối đa process chạy trong một lần dispatch. **FIFO** là thứ tự process bên trong cùng một queue.

Không nên trả lời: “Một slot bằng một instruction.” Trong implementation, một weighted slot tương ứng gần với một lần dispatch; mỗi dispatch có thể chạy nhiều instruction tùy quantum.

Câu hỏi: Khi một queue dùng hết slot nhưng process chưa xong thì sao?

Trả lời đề xuất. Lần `put_proc` đưa process về đúng priority queue. Tuy nhiên `get_mlq_proc` bỏ qua queue có `slot[prio] == 0` và thử queue khác. Khi không chọn được process nào, toàn bộ slot được reset rồi quét lại.

Mở code/bảng chứng. Hai vòng quét và khối reset trong `src/sched.c: get_mlq_proc()`. Test: `sched_out_of_slot`.

Câu hỏi: Nếu chỉ có một queue đang có process và queue đó hết slot thì sao?

Trả lời đề xuất. Vòng quét đầu không chọn được process. Scheduler reset toàn bộ slot rồi quét lần hai, vì vậy process ở queue duy nhất sẽ được chọn lại. Nó không bị mất, nhưng cũng không chuyển sang queue khác vì

thiết kế này không thay đổi priority của PCB.

Giới hạn cần nói thẳng. Đây là weighted MLQ, không phải multilevel feedback queue. Process không tự động thăng/hạ queue dựa trên hành vi CPU/I/O.

Câu hỏi: Priority cao đến trong lúc process khác đang chạy có cướp CPU ngay không?

Trả lời đề xuất. Không. CPU chỉ gọi scheduler tại scheduling boundary: lúc chưa có process, hết quantum hoặc process hoàn thành. Process priority cao mới đến sẽ ảnh hưởng lần dispatch tiếp theo, không cắt ngang instruction hiện tại.

Mở code/bằng chứng. `src/os.c: cpu_routine(); test sched_prio.`

Câu hỏi: Vì sao hai CPU không lấy cùng một PCB?

Trả lời đề xuất. `get_mlq_proc` giữ `queue_lock` trong toàn bộ đoạn dequeue khỏi ready queue và enqueue vào `running_list`. Vì vậy PCB được claim một cách nguyên tử trước khi CPU khác có thể lấy nó.

Mở code/bằng chứng. `src/sched.c: queue_lock, get_mlq_proc().`

Câu hỏi: Nếu thay đổi từ một CPU thành hai hoặc ba CPU thì chương trình có chạy không?

Trả lời đề xuất. Có. Dòng đầu config là `quantum num_cpus num_processes`. `main()` cấp mảng thread theo `num_cpus`, attach mỗi CPU vào timer, rồi tạo đúng số CPU thread. Scheduler dùng queue lock nên vẫn bảo vệ ownership.

Mở code/bằng chứng. `src/os.c: read_config(), main(); testcase` hiện có `sched_2_cpu`.

Giới hạn cần nói thẳng. Thứ tự log có thể đổi do pthread scheduling. Điều phải giữ là invariant, ví dụ không có hai CPU sở hữu cùng PID tại cùng thời điểm. Chưa có stress test dài với số CPU/process rất lớn.

Câu hỏi: Thay đổi số CPU thì output nào được phép khác?

Trả lời đề xuất. CPU nào dispatch PID nào và thứ tự các dòng độc lập có thể khác. Tổng hành vi logic phải đúng: process không mất, không chạy trùng PCB, quantum được tôn trọng và chương trình kết thúc. Không nên yêu cầu diff output tuyệt đối giữa hai lần chạy đa luồng.

Câu hỏi: Scheduler có thể starvation không?

Trả lời đề xuất. Weighted quota buộc queue ưu tiên cao dùng hết ngân sách rồi scheduler có thể xét queue thấp hơn trong cùng chu kỳ. Điều này giảm starvation so với strict priority thuần. Tuy nhiên với workload động vô hạn và thiết kế reset hiện tại, nhóm không tuyên bố đã chứng minh tuyệt đối không starvation.

4. Timer thread, concurrency và race condition

Câu hỏi: Timer thread làm gì?

Trả lời đề xuất. Timer tạo barrier theo time slot. Mỗi CPU và loader gọi `next_slot()` để báo đã hoàn thành công việc của slot hiện tại. Timer chờ tất cả device, tăng thời gian rồi signal cho chúng tiếp tục. Timer không chọn process và không thực thi instruction.

Mở code/bằng chứng. `src/timer.c: timer_routine(), next_slot(), attach_event().`

Câu hỏi: Nếu bỏ timer barrier thì chuyện gì xảy ra?

Trả lời đề xuất. CPU thread có thể chạy qua nhiều instruction trước khi loader hoặc CPU khác kịp tiến triển. Khi đó time slot mô phỏng không còn ý nghĩa và kết quả phụ thuộc mạnh vào tốc độ scheduling thread thật của hệ điều hành host.

Câu hỏi: Các lock chính bảo vệ tài nguyên gì?

Trả lời đề xuất. `queue_lock` bảo vệ ready queues và running list; `mmvm_lock` bảo vệ region allocation/free và một số thao tác memory; `memphy_lock` bảo vệ free physical frame list. Timer dùng mutex và condition variable riêng cho mỗi attached event.

Mở code/bảng chứng. `src/sched.c`, `src/libmem.c`, `src/mm-memphy.c`, `src/timer.c`.

Câu hỏi: Có thể deadlock không?

Trả lời đề xuất. Trong các đường demo chính, lock thường được lấy và thả trong phạm vi ngắn, không giữ đồng thời nhiều lock theo vòng phụ thuộc rõ ràng. Bộ test hiện tại chạy đến cuối. Tuy nhiên nhóm chưa có công cụ model checking hoặc stress test dài để chứng minh không deadlock trong mọi interleaving.

5. Logic READ, WRITE và MMU

Câu hỏi: Giải thích đầy đủ logic lệnh READ.

Trả lời đề xuất. Instruction `read source offset destination` dùng `source` như region ID, không phải raw address. `libread` đóng gói `SYSMEM_READ_OP` và gọi syscall 17. Kernel tìm caller PCB bằng PID, rồi `__read` kiểm tra region tồn tại và offset nằm trong `[start,end)`. Sau đó `pg_getval` lấy page, tính physical address, gọi physical read có kiểm tra ownership và trả byte về register destination.

```
CPU run -> libread -> syscall 17 -> __sys_mmap -> __read -> pg_getval ->
physical read
```

Mở code/bảng chứng. `src/cpu.c`, `src/libmem.c`, `src/sys_mem.c`.

Câu hỏi: Vì sao offset bằng size phải bị từ chối?

Trả lời đề xuất. Region dùng khoảng nửa mở `[start,end)`. Offset hợp lệ phải thỏa `start + offset < end`. Nếu allocation có size 50 thì offset 49 hợp lệ, offset 50 vượt biên.

Mở code/bảng chứng. Điều kiện bounds trong `__read()` và `__write()`. Test `os_mem_reuse` đọc offset 49.

Câu hỏi: Logic WRITE khác READ ở đâu?

Trả lời đề xuất. `libwrite` đóng gói region, offset và byte value vào syscall. `__write` kiểm tra region/bounds, rồi `pg_setval` lấy page và ghi byte xuống physical memory. READ trả dữ liệu qua register; WRITE nhận dữ liệu từ argument.

Mở code/bảng chứng. `src/libmem.c: libwrite(), __write(), pg_setval()`.

Câu hỏi: MMU trong simulator là gì?

Trả lời đề xuất. MMU được mô phỏng bằng các hàm dịch virtual page sang physical frame thông qua page table. Nó không phải MMU phần cứng. Với MM64, `walk_pte` đi qua PGD, P4D, PUD, PMD và PT; PTE cuối chứa trạng thái present/swapped và frame.

Câu hỏi: Vì sao phải phân biệt virtual address và physical address?

Trả lời đề xuất. Virtual address thuộc không gian riêng của process và được kiểm tra qua region/page table. Physical address là vị trí byte trong MEMRAM. Tách hai lớp giúp process không trực tiếp suy ra quyền truy cập chỉ từ một địa chỉ vật lý.

6. ALLOC, FREE, KMALLOC và kernel cache

Câu hỏi: ALLOC hoạt động như thế nào?

Trả lời đề xuất. `liballoc` gửi `SYSTEMEM_ALLOC_OP`. Kernel gọi `__alloc`. Hàm này ưu tiên tìm một free logical region đủ lớn; nếu không có thì tăng VMA bằng `inc_vma_limit`, map thêm page/frame và lưu region vào `symrgtbl[rgid]`. Địa chỉ bắt đầu được trả về register tương ứng.

Mở code/bằng chứng. `src/libmem.c: liballoc(), __alloc(), get_free_vmrg_area(); src/mm-vm.c: inc_vma_limit().`

Câu hỏi: FREE có trả physical frame ngay không?

Trả lời đề xuất. Đối với user logical region trong implementation này, `__free` đưa khoảng region vào free-region list và xóa symbol/register. Nó chứng minh logical region reuse, không nên kết luận mọi physical frame đã được unmap và trả ngay tại thời điểm FREE. Khi process kết thúc, `destroy_mm` giải phóng user frames và page-table tree.

Mở code/bằng chứng. `src/libmem.c: __free(); src/mm64.c: destroy_mm().`

Câu hỏi: Phân biệt ALLOC và KMALLOC.

Trả lời đề xuất. ALLOC cấp logical user region trong VMA và dùng page-table mapping. KMALLOC cấp kernel region bắt đầu từ `PAGING_KMEM_BASE`, yêu cầu các physical frame liên tiếp và lưu metadata trong `ksymrgtbl`, `kphy_start`, `kphy_npages`.

Mở code/bằng chứng. `src/libmem.c: __alloc()` so với `__kmalloc()`; `include/os-mm.h`.

Giới hạn cần nói thẳng. Đây là mô hình đơn giản, không tương đương đầy đủ Linux `kmalloc`.

Câu hỏi: Kernel cache pool là gì?

Trả lời đề xuất. Cache pool cấp trước một vùng kernel bằng `__kmalloc`, chia thành các slot cùng kích thước `align`. `__kmem_cache_alloc` trả lần lượt từng slot bằng `storage + used * align`. Nó mô phỏng ý tưởng slab/cache cho object cùng kích thước.

Mở code/bằng chứng. `src/libmem.c: __kmem_cache_pool_create(), __kmem_cache_alloc().`

Câu hỏi: Copy from user và copy to user hoạt động ra sao?

Trả lời đề xuất. `copy_from_user` đọc từng byte từ user region bằng `__read_user_mem`, rồi ghi vào kernel region. Nếu kernel destination chưa tồn tại, hàm cấp bằng `__kmalloc`. `copy_to_user` làm chiều ngược lại. Test roundtrip ghi 65 ở user, copy vào kernel, copy sang user region khác rồi đọc lại 65.

Mở code/bằng chứng. `src/libmem.c: __kmem_copy_from_user(), __kmem_copy_to_user(); test os_mem_roundtrip_spec.`

7. Syscall, wrapping và user/kernel protection

Câu hỏi: Wrapping là gì trong bài này?

Trả lời đề xuất. Wrapper là hàm phía thư viện chuyển lời gọi mức instruction thành syscall với các scalar arguments. Ví dụ `liballoc`, `libread`, `libkmem_copy_from_user` chuẩn bị `sc_regs` rồi gọi syscall 17. Wrapper che chi tiết dispatcher và tạo một giao diện thống nhất.

Câu hỏi: Tại sao syscall không truyền trực tiếp PCB pointer?

Trả lời đề xuất. User side không nên được phép đưa một `pcb_t*` tùy ý cho kernel. Wrapper chỉ truyền PID và scalar arguments. Kernel dùng `find_proc_by_pid` để lấy PCB thật trong registry scheduler. Đây là trust boundary đơn giản của simulator.

Mở code/bằng chứng. `src/libstd.c: libsyscall(); src/sys_mem.c: __sys_mmap(); src/sched.c: find_proc_by_pid().`

Câu hỏi: Syscall table được tạo và dispatch thế nào?

Trả lời đề xuất. `src/syscall.tbl` ánh xạ number sang entry point. Script `syscalltbl.sh` sinh `syscalltbl.lst`. `_syscall` dùng macro để tạo switch và gọi handler tương ứng. Syscall memory của bài là số 17.

Mở code/bằng chứng. `src/syscall.tbl, src/syscall.c, Makefile.`

Câu hỏi: Biết physical address có được đọc frame không?

Trả lời đề xuất. Không. Với operation physical READ/WRITE, kernel gọi `caller_owns_phyaddr`. Trong MM64, hàm này đi qua page-table tree của caller và chỉ cho phép nếu một present PTE của caller trở tới frame đó.

Mở code/bằng chứng. `src/sys_mem.c: caller_owns_phyaddr(); src/mm64.c: paging64_owns_fpn(); test os_cross_pid_protection.`

Câu hỏi: Phân biệt translation và authorization.

Trả lời đề xuất. Translation trả lời “virtual address này nằm ở physical frame nào”. Authorization trả lời “caller có quyền dùng frame đó không”. Một địa chỉ dịch được hoặc bị lộ không tự động tạo quyền truy cập.

Câu hỏi: Bài có kiểm tra user/kernel address range hoàn chỉnh không?

Trả lời đề xuất. Bài tách user regions trong `symrgtbl` và kernel regions trong `ksymrgtbl`, kernel allocation bắt đầu từ `PAGING_KMEM_BASE`, và copy đi qua các helper riêng. Tuy nhiên nhóm không tuyên bố đây là cơ chế privilege/range protection đầy đủ như kernel thật; đây là mô hình hóa bằng metadata, wrappers và ownership check.

8. Paging 64-bit, canonical address và swap

Câu hỏi: Tại sao cần multi-level paging?

Trả lời đề xuất. Một flat page table cho không gian địa chỉ lớn sẽ tốn bộ nhớ rất lớn dù process chỉ dùng ít địa chỉ. Multi-level paging chỉ cấp các bảng cho branch được chạm, đổi lại mỗi translation có thể cần nhiều

lần truy cập page table.

Câu hỏi: Địa chỉ 64-bit được chia như thế nào?

Trả lời đề xuất. Trong mô hình MM64, page size là 4 KiB nên offset dùng 12 bit. Page number được chia thành năm index 9 bit: PGD, P4D, PUD, PMD, PT. Mỗi bảng có 512 entry và kích thước 4096 byte nếu mỗi entry là 8 byte.

Mở code/bảng chứng. `include/mm64.h; src/mm64.c: get_pd_from_address()`.

Câu hỏi: Canonical address là gì?

Trả lời đề xuất. Không phải mọi mẫu 64-bit đều hợp lệ. Với địa chỉ 57-bit của mô hình, bit 63–57 phải là sign extension của bit 56. `paging64_is_canonical` kiểm tra điều này trước khi walk hoặc cấp bảng.

Mở code/bảng chứng. `src/mm64.c: paging64_is_canonical(), vmap_pgd_memset()`.

Câu hỏi: walks, accesses và bytes nghĩa là gì?

Trả lời đề xuất. `walks` tăng một lần mỗi khi gọi page-table walk. `accesses` tăng theo số cấp table thực sự được truy cập trong walk. `bytes` tăng khi `alloc_table` cấp một bảng sparse mới. Một walk đủ năm cấp thường làm `accesses` tăng năm, nhưng walk dừng sớm có thể ít hơn.

Mở code/bảng chứng. `src/mm64.c: walk_pte(), alloc_table(), print_pgtbl()`.

Câu hỏi: Page fault được mô phỏng ở đâu?

Trả lời đề xuất. Khi `pg_getpage` thấy PTE không present, nó tìm victim bằng FIFO, lấy swap frame, copy victim từ RAM sang swap, cập nhật PTE, rồi nếu target từng ở swap thì copy target trở lại RAM. Đây là đường xử lý tương tự page fault trong mô hình.

Mở code/bảng chứng. `src/libmem.c: pg_getpage(), find_victim_page(); src/mm-vm.c: __mm_swap_page(), __mm_swap_page_in()`.

Câu hỏi: Swap đã hoàn chỉnh và demo được chưa?

Trả lời đề xuất. Code path swap-out/swap-in có tồn tại: FIFO victim, copy page giữa MEMRAM và MEM-SWAP, cập nhật PTE. Tuy nhiên bộ demo hiện tại chưa cô lập được một kịch bản end-to-end mạnh chứng minh dữ liệu swap-out rồi swap-in vẫn đúng. Vì vậy nhóm nên nói “đã có implementation path nhưng chưa có bằng chứng demo đủ mạnh”, không nói swap hoàn chỉnh tuyệt đối.

Không nên trả lời: “Swap chắc chắn hoàn chỉnh vì code có hàm swap.” Có code path không đồng nghĩa đã kiểm chứng mọi trường hợp.

Câu hỏi: MEMRAM và MEMSWAP có phải RAM/ổ đĩa thật không?

Trả lời đề xuất. Không. Chúng là `memphy_struct` chứa mảng byte và free-frame list. MEMRAM/MEM-SWAP mô phỏng thiết bị nhớ trong chương trình C.

Mở code/bảng chứng. `src/mm-memphy.c; include/os-mm.h`.

9. Input, output, assertions và thay đổi trực tiếp

Câu hỏi: Giải thích dòng đầu của configuration.

Trả lời đề xuất. Dòng đầu là `quantum num_cpus num_processes`. Ví dụ `2 2 3` nghĩa là quantum 2 instruction mỗi dispatch, hai CPU thread và ba process sẽ được loader tạo. Với paging config mới, dòng tiếp theo có kích thước MEMRAM và các MEMSWAP.

Mở code/bằng chứng. `src/os.c: read_config()`.

Câu hỏi: Nếu thay đổi input, nhóm kiểm tra gì trước khi chạy?

Trả lời đề xuất. Kiểm tra số process ở dòng đầu khớp số dòng process; tên process file tồn tại; priority nằm trong 0–139; instruction count trong process file khớp số dòng; memory size đủ; các region ID nằm trong bảng symbol. Sau đó dự đoán property và output cần quan sát trước khi chạy.

Câu hỏi: Tại sao output giữa Linux và macOS hoặc giữa hai lần chạy có thể khác?

Trả lời đề xuất. Loader và CPU là pthread, nên thứ tự các event độc lập có thể khác theo scheduler của host. Semantic invariant phải giống: không chạy trùng PCB, dữ liệu đúng, quyền truy cập đúng, canonical address bị kiểm tra và chương trình kết thúc.

Câu hỏi: Assertion là gì?

Trả lời đề xuất. Assertion là kiểm tra tự động một property bắt buộc. Script `tests/assert_outputs.sh` không yêu cầu toàn bộ log giống tuyệt đối; nó tìm các bằng chứng semantic như cross-PID bị deny, region được reuse, byte 65 roundtrip và non-canonical address bị reject.

Mở code/bằng chứng. `tests/assert_outputs.sh`.

Câu hỏi: Test pass chứng minh được gì và không chứng minh được gì?

Trả lời đề xuất. Nó chứng minh source build được, các testcase chạy đến cuối và các property được assert xuất hiện trong output. Nó không chứng minh mọi input, mọi race interleaving, mọi trường hợp swap-full hoặc mọi lỗi memory safety đều đã được xử lý.

9.1. Các tình huống thầy có thể yêu cầu sửa tại chỗ

Yêu cầu	Dự đoán trước khi chạy	Điều cần giải thích
Đổi 1 CPU thành 2/3 CPU	Các event độc lập có thể đổi thứ tự; throughput mô phỏng tăng; PCB ownership phải duy nhất.	<code>num_cpus</code> , <code>thread creation</code> , <code>queue_lock</code> , timer attachment.
Đổi quantum từ 2 thành 1	CPU gọi <code>put_proc</code> thường xuyên hơn; nhiều scheduling boundary hơn.	Quantum khác weighted slot; mỗi dispatch chỉ chạy tối đa một instruction.
Cho hai process cùng priority	Trong cùng queue, thứ tự là FIFO theo lúc loader enqueue.	<code>enqueue/dequeue</code> và nondeterminism nếu load cùng slot.
Đổi priority process	Queue được xét thay đổi; số weighted slot cũng thay đổi theo <code>140-prio</code> .	Priority số nhỏ được quét trước nhưng có quota lớn hơn theo công thức hiện tại.
READ offset bằng size	Phải trả lỗi vì region là <code>[start, end)</code> .	Chỉ điều kiện bounds trong <code>__read</code> .

Yêu cầu	Dự đoán trước khi chạy	Điều cần giải thích
PID 2 đọc frame PID 1	Kernel phải in denied.	Translation khác authorization; caller_owns_phyaddr.
Địa chỉ MM64 non-canonical	Phải bị reject trước khi walk/cấp bảng; thống kê bytes không tăng vì mapping đó.	paging64_is_canonical.
Giảm RAM rất nhỏ	Allocation có thể fail; swap path chỉ kích hoạt khi truy cập page không present, không phải mọi allocation tự động swap.	Nói trước giới hạn và không hứa demo swap nếu chưa chuẩn bị testcase.

10. Câu hỏi về quá trình làm và sử dụng AI

Câu hỏi: Nhóm có sử dụng AI không?

Trả lời đề xuất. Có thể trả lời trung thực: “Nhóm có sử dụng AI như công cụ hỗ trợ đọc codebase cũ, gợi ý testcase, rà lỗi và hỗ trợ tài liệu. Nhóm không coi output của AI là bằng chứng đúng. Mỗi thay đổi được đối chiếu với đặc tả, đọc lại code, build từ sạch và kiểm tra bằng testcase/semantic assertions.”

Câu hỏi: AI được dùng ở khâu nào?

Trả lời đề xuất. Nêu đúng những khâu nhóm thực sự đã dùng, ví dụ: gợi ý cách tổ chức báo cáo/demo; hỗ trợ tìm luồng call; đề xuất negative test; rà code và giải thích trade-off. Không nên kể AI tự làm toàn bộ implementation nếu điều đó không phản ánh quá trình thật.

Câu hỏi: Làm sao biết AI đúng hay sai?

Trả lời đề xuất. Không tin theo câu trả lời đơn lẻ. Nhóm kiểm chứng bằng bốn lớp:

1. đối chiếu đặc tả và source gốc;
2. đọc diff theo từng file và giải thích được từng thay đổi;
3. tạo testcase cô lập property, gồm cả negative test;
4. clean build, chạy semantic assertions và thử thay đổi input.

Mở code/bằng chứng. `git diff, make clean && make all, make test, tests/assert_outputs.sh.`

Câu hỏi: Một ví dụ AI/gợi ý có thể sai và nhóm xử lý thế nào?

Trả lời đề xuất. Ví dụ dễ nói: yêu cầu output đa luồng khớp tuyệt đối là không hợp lý vì pthread ordering có thể đổi. Nhóm chuyển từ so sánh toàn bộ log sang semantic assertions kiểm tra invariant. Hoặc khi cú pháp copy trong đặc tả ngắn hơn test nội bộ, nhóm đọc loader và bổ sung parser tương thích rồi tạo testcase `os_mem_roundtrip_spec`.

Câu hỏi: Nếu thầy hỏi “đoạn này ai viết” thì trả lời sao?

Trả lời đề xuất. Trả lời đúng phân công và quá trình review thực tế. Người phụ trách phải giải thích được mục tiêu, input/output, các hàm liên quan và giới hạn. Nếu một đoạn được hỗ trợ bởi AI hoặc thành viên khác, nói rõ nhóm đã review và test nó như thế nào; không nhận sai tác giả và không né câu hỏi kỹ thuật.

11. Các câu hỏi khó và cách thừa nhận giới hạn

Câu hỏi: Tại sao codebase thay đổi thiết kế so với source gốc?

Trả lời đề xuất. Một số thay đổi mở rộng thiết kế để đáp ứng yêu cầu và kiểm chứng rõ hơn: PCB có mm riêng; syscall memory dùng PID lookup; MM64 dùng sparse page-table tree; scheduler có running list bảo vệ multi-CPU. Nhóm phải chỉ ra được yêu cầu/property mà mỗi thay đổi phục vụ và trade-off của nó.

Giới hạn cần nói thẳng. Thiết kế mở rộng làm code phức tạp hơn source khung. Nhóm không nên khẳng định đây là cách duy nhất hoặc tối ưu nhất.

Câu hỏi: Phần nào có nguy cơ over-engineer?

Trả lời đề xuất. Unified syscall nhiều operation, kernel cache helpers, MM64 sparse tree và bộ test mở rộng vượt mức TODO tối thiểu. Chúng có giá trị vì giúp mô phỏng trust boundary và tạo bằng chứng; nhưng tăng bề mặt code, chi phí review và rủi ro bug. Khi bảo vệ, tập trung vào property cần thiết thay vì khoe số lượng tính năng.

Câu hỏi: Điểm yếu kỹ thuật lớn nhất hiện tại là gì?

Trả lời đề xuất. Có thể nêu ba điểm: swap chưa có demo end-to-end mạnh; chưa có stress test race/swap-full/input lỗi dài; một số lookup/lock là global và tuyến tính nên không scale. Nên kèm hướng cải thiện: testcase ép swap, sanitizer/stress, per-structure lock hoặc PID map.

Câu hỏi: Nếu demo bất ngờ fail thì xử lý thế nào?

Trả lời đề xuất. Không chạy lại vô thức. Đọc lỗi và phân loại:

- build fail: kiểm tra generated syscall list và compiler;
- output ordering khác: kiểm tra invariant thay vì diff tuyệt đối;
- assertion fail: mở input và output tương ứng, lần theo call-flow;
- hang: nghi barrier/deadlock, xem CPU/loader nào chưa detach hoặc `next_slot`.

12. Bảng mở source trong 10 giây

Thầy hỏi	Mở file/hàm đầu tiên	Sau đó mở
Luồng toàn bộ simulator	<code>src/os.c: main(),</code> <code>cpu_routine(), ld_routine()</code>	<code>src/loader.c, src/cpu.c</code>
MLQ/weighted slot	<code>src/sched.c: get_mlq_proc()</code>	<code>src/queue.c, input scheduler</code>
Hai/ba CPU	<code>src/os.c: main()</code>	<code>src/sched.c: queue_lock,</code> <code>running_list</code>

Thầy hỏi	Mở file/hàm đầu tiên	Sau đó mở
Timer/barrier	src/timer.c: timer_routine(), next_slot()	các lời gọi next_slot trong src/os.c
Logic READ	src/libmem.c: libread(), __read(), pg_getval()	src/sys_mem.c: SYSMEM_READ/IO_READ
Logic WRITE	src/libmem.c: libwrite(), __write(), pg_setval()	src/sys_mem.c: SYSMEM_WRITE/IO_WRITE
ALLOC/FREE	src/libmem.c: __alloc(), __free()	src/mm-vm.c: inc_vma_limit()
ALLOC vs KMALLOC	src/libmem.c: __alloc(), __kmalloc()	include/os-mm.h
Wrapping/syscall	src/libstd.c, src/syscall.c	src/sys_mem.c, src/syscall.tbl
PID protection	src/sys_mem.c: caller_owns_phyaddr()	src/sched.c: find_proc_by_pid()
MM64/canonical	src/mm64.c: paging64_is_canonical(), walk_pte()	include/mm64.h
Swap	src/libmem.c: pg_getpage()	src/mm-vm.c: __mm_swap_page()
Assertions	tests/assert_outputs.sh	input và output testcase tương ứng

13. Bộ câu hỏi luyện nhanh cho từng thành viên

13.1. Bảo Anh – kiến trúc và kiểm chứng

1. Simulator khác OS thật ở điểm nào?
2. Loader, scheduler, CPU, timer, syscall và MMU nối với nhau ra sao?
3. Vì sao test pass không đồng nghĩa không còn bug?
4. Nhóm dùng AI ở đâu và kiểm chứng thế nào?
5. Ba giới hạn lớn nhất của bài là gì?

13.2. Quốc Trung – scheduler và synchronization

1. Phân biệt priority, slot, quantum và FIFO.
2. Vì sao hai CPU không lấy cùng PCB?
3. Một queue duy nhất hết slot thì chuyện gì xảy ra?
4. Timer barrier ngăn lỗi mô phỏng nào?
5. Nếu đổi thành ba CPU, invariant nào phải giữ?

13.3. Quốc Bảo – memory và copy

1. Giải thích từng bước của READ và WRITE.
2. FREE trả logical region hay physical frame?
3. ALLOC khác KMALLOC thế nào?
4. Copy from user/to user bảo toàn dữ liệu thế nào?
5. Vì sao offset bằng size là lỗi?

13.4. Quốc An – syscall và protection

1. Wrapper và syscall dispatcher khác nhau thế nào?
2. Vì sao không truyền PCB pointer từ user?
3. Translation khác authorization thế nào?
4. Cross-PID protection kiểm tra ownership ở đâu?
5. User/kernel separation của bài mạnh đến đâu?

13.5. Trường Giang – MM64, paging và swap

1. Vì sao cần multi-level paging?
2. Canonical address là gì?
3. walks/accesses/bytes nghĩa là gì?
4. Page fault và FIFO victim diễn ra thế nào?
5. Swap đã được chứng minh end-to-end chưa?

14. Checklist trước khi bước vào phòng demo

- Mỗi thành viên chạy được testcase thuộc phần mình và giải thích input trước khi nhìn output.
- Mỗi thành viên mở được file/hàm chính trong dưới 10 giây.
- Cả nhóm thuộc lòng: loader → PCB → scheduler → CPU → syscall/MMU → MEMRAM/MEMSWAP.
- Chuẩn bị sẵn câu trả lời trung thực về AI, swap và các giới hạn.
- Chạy trước:

```
make clean && make all
make test
./os sched_prio
./os sched_out_of_slot
./os sched_2_cpu
./os os_mem_roundtrip_spec
./os os_cross_pid_protection
./os os_mm64_canonical
```

15. Cách xử lý khi bị hỏi ngoài kịch bản

Câu hỏi: Nếu chưa nhớ chính xác tên hàm hoặc chưa chắc câu trả lời thì làm gì?

Trả lời đề xuất. Không đoán bừa. Nói rõ phần đang chắc chắn ở mức property, sau đó xin mở source để lần theo call path. Ví dụ: “Em chưa nhớ chính xác tên helper, nhưng READ bắt buộc phải kiểm tra region và offset trước khi dịch sang physical address. Em xin mở từ `libread()` để chỉ đúng hàm thực hiện từng bước.” Cách trả lời này thể hiện hiểu thiết kế, đồng thời tránh khẳng định sai chi tiết.

Câu hỏi: Nếu output trực tiếp khác output đã chuẩn bị thì trả lời thế nào?

Trả lời đề xuất. Trước tiên phân biệt khác thứ tự log do pthread với sai property. Nhóm kiểm tra các invariant như không dispatch cùng PCB cho hai CPU, giá trị đọc đúng, truy cập trái PID bị từ chối và địa chỉ non-canonical bị reject. Nếu invariant sai, nhóm thừa nhận đây là lỗi cần điều tra; nếu chỉ thứ tự log khác nhưng invariant giữ, nhóm giải thích nguyên nhân nondeterminism của host scheduler.

Câu hỏi: Nếu thấy yêu cầu chỉ ra một lỗi từng gặp trong quá trình làm?

Trả lời đề xuất. Trình bày theo bốn bước: triệu chứng quan sát được; giả thuyết nguyên nhân; test nhỏ dùng để cô lập lỗi; thay đổi code và assertion ngăn tái phát. Không cần kể câu chuyện hoàn hảo. Một câu trả lời tốt phải cho thấy nhóm biết chuyển từ output bất thường thành property có thể kiểm tra.

Câu hỏi: Nếu phát hiện một phần chưa được chứng minh đầy đủ ngay trong buổi demo?

Trả lời đề xuất. Nói thẳng phạm vi bằng chứng hiện có. Ví dụ với swap: “Source có đường page replacement và swap-in/swap-out, nhưng testcase hiện tại chưa ép đủ áp lực để chứng minh toàn bộ vòng đời end-to-end. Vì vậy nhóm chỉ khẳng định phần code path và giới hạn bằng chứng.” Thừa nhận đúng giới hạn đáng tin cậy hơn việc suy diễn từ một dòng output.

Câu kết luận nên nhớ:

“Nhóm không chỉ dựa vào output. Nhóm xác định property cần giữ, chỉ ra code giữ property đó, thiết kế input cô lập nó, kiểm tra output bằng assertion và nói rõ những trường hợp chưa được chứng minh.”